

Tabby局限性

1. 调用图规模与搜索效率问题

现有反序列化漏洞检测工具（如 TABBY）在调用图构建阶段依赖基于类层次分析的保守近似，以覆盖 Java 运行时多态行为。然而，该策略在反序列化场景下会引入大量不可达的虚假调用边，导致调用图规模显著膨胀。随着过程间分析的展开，冗余路径被不断传播并放大，增加了 gadget 链搜索的复杂度，不仅影响分析效率，也降低了检测结果的精确性。

2. 反射与动态代理建模不足

此外，TABBY 作为静态控制流分析方法，在处理包含反射与动态代理的调用路径时仍存在局限。由于相关调用目标在运行时动态确定，静态分析难以精确建模真实的执行语义，分析过程只能依赖保守近似，从而在复杂反序列化利用场景中产生不准确或不完整的调用关系，进一步限制了检测精度。

根据他论文中构建调用图所示

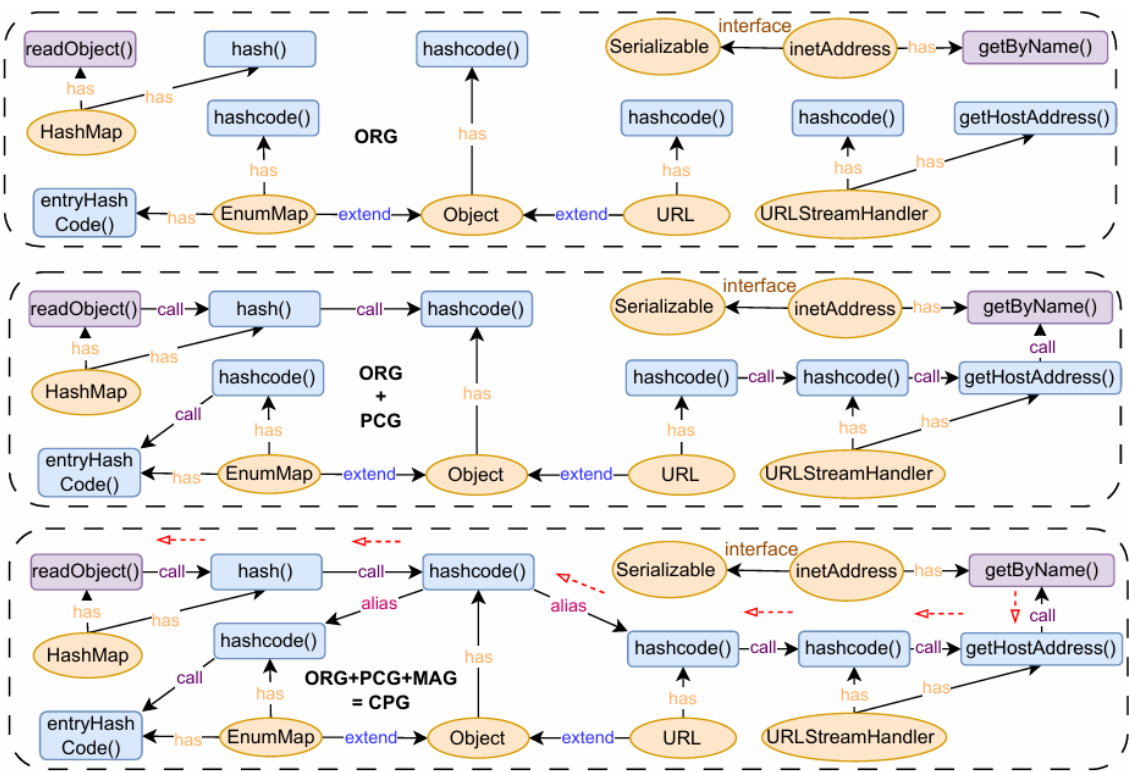


Fig. 4. Example of Code Property Graph

他采取全量调用图的构建，这样会导致调用图十分大，他不想flash那种，从readObject这个source开始构建，而是采取类似CHA的算法，管它这那的，把可能的边都加上，后续在做他所谓的污点分析。所以我们可以说他构建的图的边数很大，但是虚假边很多，我们在此基础上改进。

然后它实验部分给出的调用边数看起来很小，但是实际跑下来差点很多很多。边数太多，这个也可以和flash对比，它全量构建的调用图，竟然能比flash构建调用图规模还小，是不合理的

TABLE VIII
EXPERIMENTAL DATA UNIT INFORMATION

Code amount (MB)	Jar file count	Class node count	Method node count	Relationship Edge count	Time consuming (min)
10	29	9055	59508	189021	1.9
20	63	14765	107623	341111	3.1
30	88	21104	153653	491651	6.0
40	93	25532	198130	628392	9.8
50	95	30859	249545	816421	12.7
100	113	32713	268670	857881	20.1
150	155	66247	503358	1587266	36.3

我在跑大概12000个class的分析时候，就已经报OOM了，也有可能是我电脑不行，但是我之前有一次机缘巧合跑出来过一次，他那个边数我记得是1440000条边，达百万条了。

第二个是他没有处理反射和动态代理，会有误报，但是由于它上面构建的是全量的调用图，从理论上来说，漏报不会太多，但是跑下来，其实并没有他实验中表达的效果那么好，其实也是，flash都没那么好的效果，这个也挺悬。然后我们也可以说，tabby没有处理java的动态语言特性，有漏报给了我们改进的空间